

LABORATORY OBSERVATION / RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	4
Student Name	Kottana Roshini
Roll No.	A24126552262

1. TITLE

Differentiate Between JavaScript Functions and Classes and Create Multiple Objects Using a Class

2. AIM

To understand and differentiate JavaScript functions and classes, and to demonstrate how a class can be used to create multiple objects with common properties and methods.

3. OBJECTIVE

- Understand functions in JavaScript.
- Understand classes and objects in JavaScript.
- Differentiate functions and classes.
- Define common properties and methods inside a class.
- Create multiple objects from the same class.
- Access object properties and invoke common methods.
- Verify the program using multiple test cases.

CO-1: Students learn object-oriented programming concepts in JavaScript, including classes, objects, properties, and methods.

4. THEORY

JavaScript Function: A function is a reusable block of code designed to perform a particular task. It can accept parameters, perform operations, and return a value.

JavaScript Class: A class is a blueprint for creating objects. It defines common properties and methods that objects created from the class can use.

Object: An object is an instance of a class. Multiple objects can be created from one class, and each object can contain different property values while sharing the same methods.

Constructor: The constructor() method initializes object properties when a new object is created.

Difference Between Functions and Classes

Function	Class
A reusable block of code that performs a task.	A blueprint used to create objects.
Can be called directly.	Objects are generally created using the new keyword.
Can accept parameters and return values.	Can define properties and methods for its objects.
Mainly used for reusable operations or behavior.	Mainly used to model objects and their common behavior.

5. ALGORITHM / PROCEDURE

1. Create a new JavaScript program.
2. Define a simple function.
3. Define a Student class using the class keyword.

4. Create a constructor to initialize name, roll number, and marks.
5. Define displayDetails() to display student information.
6. Create multiple Student objects using new.
7. Assign different property values to each object.
8. Call displayDetails() for each object.
9. Run the program and observe the output.
10. Compare expected and actual results for multiple test cases.

6. PROGRAM

```
// JavaScript Functions and Classes

function greet(name) {
    return "Hello, " + name + "!";
}

class Student {
    constructor(name, rollNo, marks) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
    }

    displayDetails() {
        return "Name: " + this.name +
            "\nRoll No: " + this.rollNo +
            "\nMarks: " + this.marks;
    }
}

const student1 = new Student("Ravi", "101", 85);
const student2 = new Student("Priya", "102", 92);
const student3 = new Student("Arjun", "103", 67);
const student4 = new Student("Kiran", "104", 0);
const student5 = new Student("Meena", "105", 100);

console.log(greet("Student"));
console.log("\nStudent 1");
console.log(student1.displayDetails());
console.log("\nStudent 2");
console.log(student2.displayDetails());
console.log("\nStudent 3");
console.log(student3.displayDetails());
console.log("\nStudent 4");
console.log(student4.displayDetails());
console.log("\nStudent 5");
console.log(student5.displayDetails());
```

7. CODE EXPLANATION

Code	Explanation
function greet(name)	Defines a reusable JavaScript function that accepts a name and returns a greeting.
class Student	Defines a blueprint for creating Student objects.
constructor(name, rollNo, marks)	Initializes the common properties of every Student object.
this.name, this.rollNo, this.marks	Stores values separately for each object.
displayDetails()	Defines a common method used by all Student objects.
new Student(...)	Creates a new object from the Student class.
student1, student2, student3...	Different objects created from the same class.

8. TEST CASES AND EXECUTION DATA

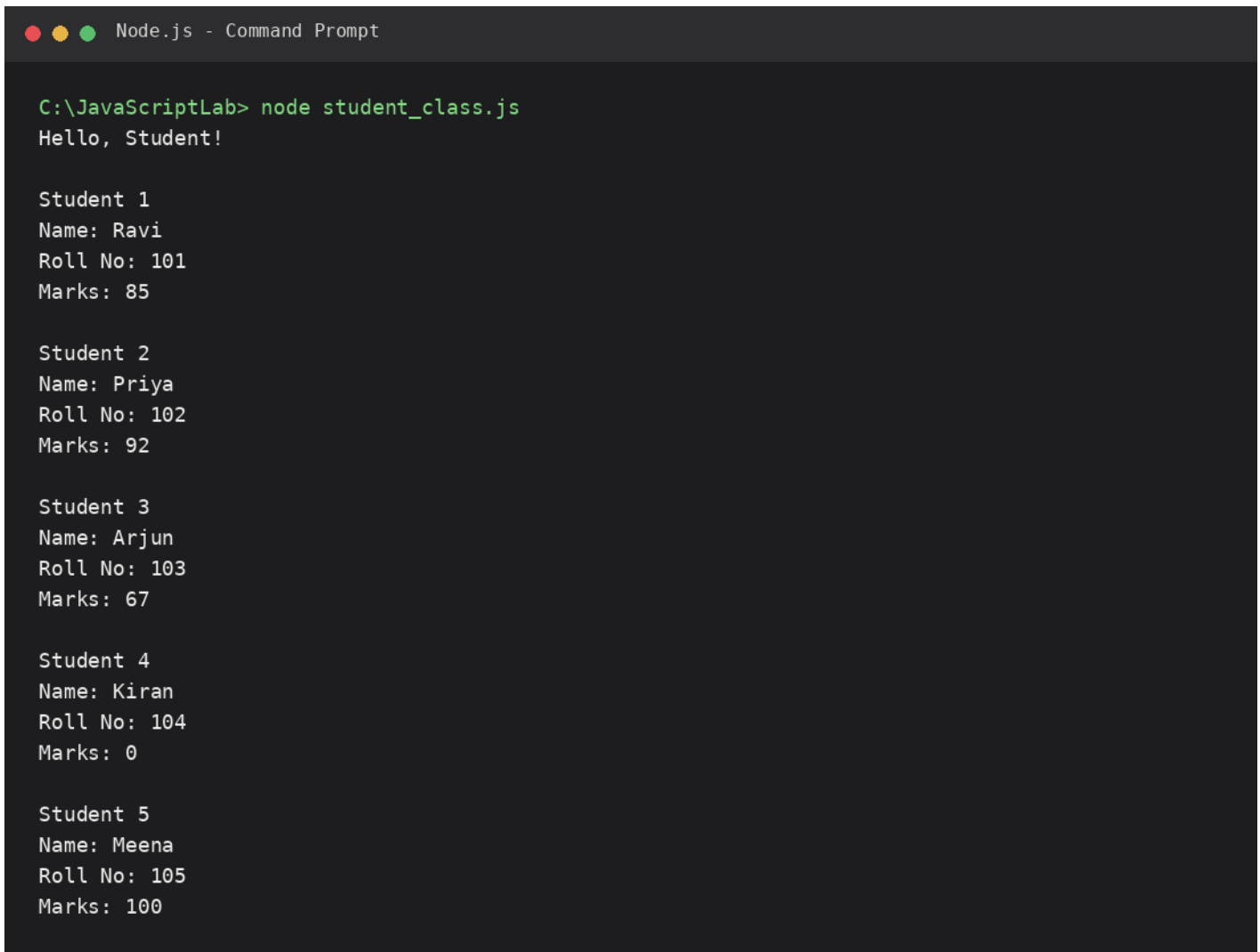
These test cases verify that multiple objects can be created from one class and that each object retains its own property values while using the common method.

Test Case	Name	Roll No.	Marks	Expected Result	Actual Result	Status
TC-01	Ravi	101	85	Student details displayed correctly	Student details displayed correctly	Pass
TC-02	Priya	102	92	Student details displayed correctly	Student details displayed correctly	Pass
TC-03	Arjun	103	67	Student details displayed correctly	Student details displayed correctly	Pass
TC-04	Kiran	104	0	Student details displayed correctly	Student details displayed correctly	Pass
TC-05	Meena	105	100	Student details displayed correctly	Student details displayed correctly	Pass

9. OUTPUT - EXECUTION SCREENSHOTS

The execution screenshots are attached on the following pages and match the recorded test-case outputs.

Output 1: Node.js program execution in the command prompt



```
Node.js - Command Prompt

C:\JavaScriptLab> node student_class.js
Hello, Student!

Student 1
Name: Ravi
Roll No: 101
Marks: 85

Student 2
Name: Priya
Roll No: 102
Marks: 92

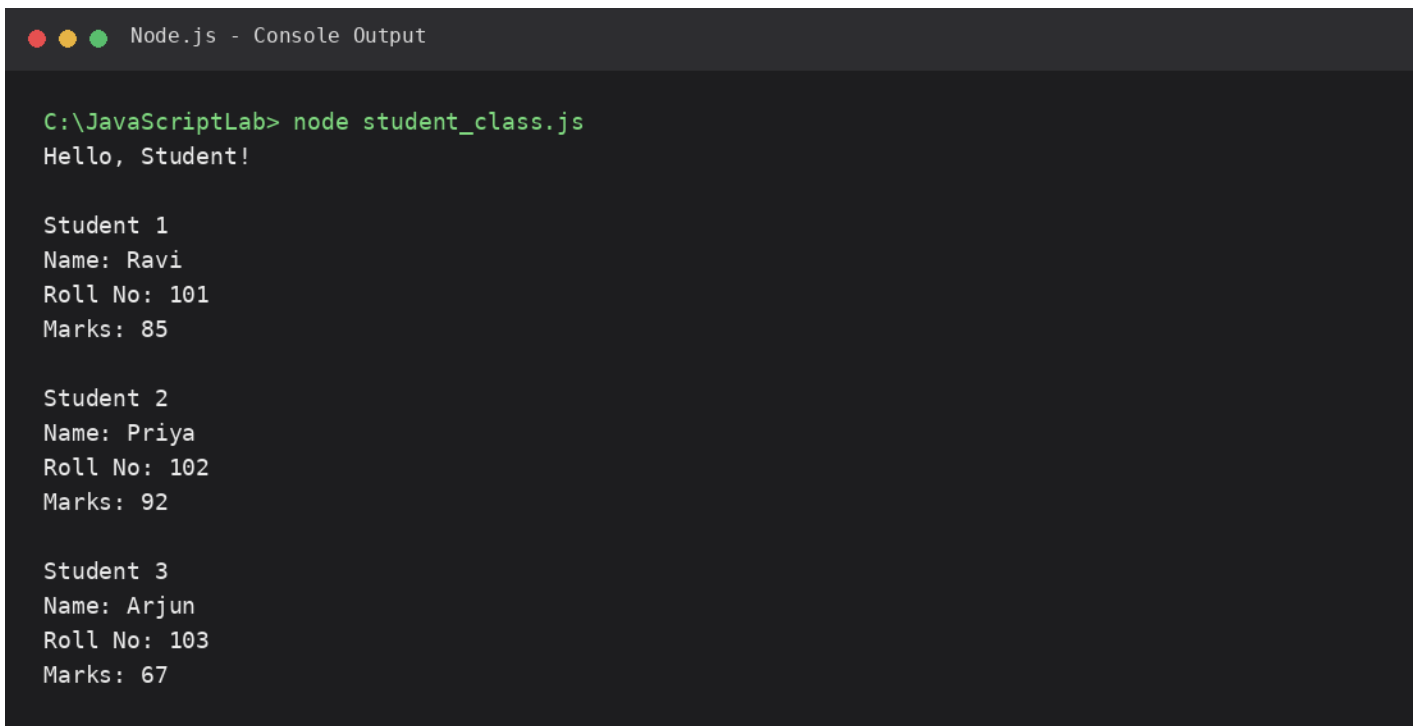
Student 3
Name: Arjun
Roll No: 103
Marks: 67

Student 4
Name: Kiran
Roll No: 104
Marks: 0

Student 5
Name: Meena
Roll No: 105
Marks: 100
```

The screenshot shows a Windows Command Prompt window titled "Node.js - Command Prompt". The prompt is at "C:\JavaScriptLab>". The user has entered the command "node student_class.js". The program has executed and produced the following output: "Hello, Student!". It then lists five students with their details: Student 1 (Ravi, Roll No: 101, Marks: 85), Student 2 (Priya, Roll No: 102, Marks: 92), Student 3 (Arjun, Roll No: 103, Marks: 67), Student 4 (Kiran, Roll No: 104, Marks: 0), and Student 5 (Meena, Roll No: 105, Marks: 100).

Output 2: Matching console output for Test Cases TC-01 to TC-03



```
Node.js - Console Output

C:\JavaScriptLab> node student_class.js
Hello, Student!

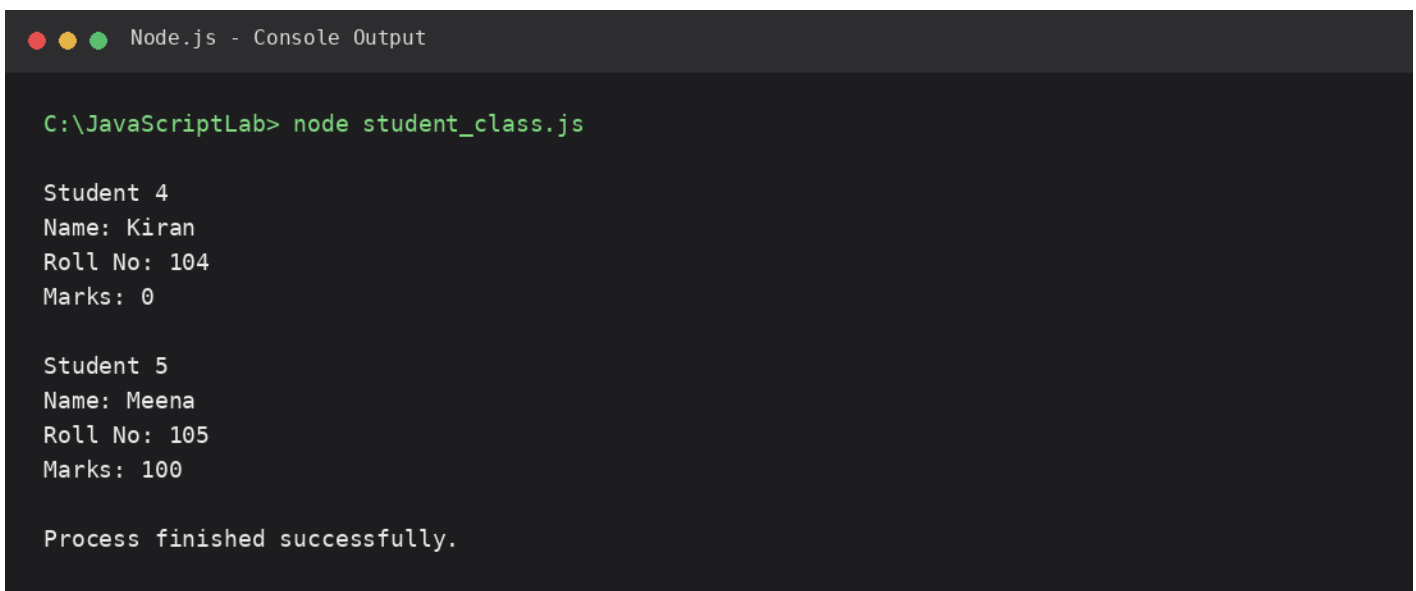
Student 1
Name: Ravi
Roll No: 101
Marks: 85

Student 2
Name: Priya
Roll No: 102
Marks: 92

Student 3
Name: Arjun
Roll No: 103
Marks: 67
```

The screenshot shows a terminal window titled "Node.js - Console Output". The prompt is "C:\JavaScriptLab>". The command executed is "node student_class.js". The output is "Hello, Student!". Below this, there are three student records. Each record starts with "Student X" (where X is 1, 2, or 3), followed by "Name: [Name]", "Roll No: [Roll Number]", and "Marks: [Marks]". The records are: Student 1 (Ravi, 101, 85), Student 2 (Priya, 102, 92), and Student 3 (Arjun, 103, 67).

Output 3: Matching console output for Test Cases TC-04 and TC-05



```
Node.js - Console Output

C:\JavaScriptLab> node student_class.js

Student 4
Name: Kiran
Roll No: 104
Marks: 0

Student 5
Name: Meena
Roll No: 105
Marks: 100

Process finished successfully.
```

The screenshot shows a terminal window titled "Node.js - Console Output". The command `C:\JavaScriptLab> node student_class.js` has been executed. The output displays information for two students: Student 4 (Kiran, Roll No: 104, Marks: 0) and Student 5 (Meena, Roll No: 105, Marks: 100). The process concludes with the message "Process finished successfully."

10. RESULT

Thus, JavaScript functions and classes were successfully differentiated. A Student class was created with common properties and a common method, and multiple Student objects were created from the same class with different property values. The program was executed successfully and the actual outputs matched the expected outputs for all five test cases.

11. CONCLUSION

This experiment demonstrates the basic object-oriented programming concepts of JavaScript. A class provides a reusable structure for creating multiple objects, while each object maintains its own data and shares the methods defined by the class.